2 Starting a project

This chapter covers

- Introducing the Hibernate and Spring Data projects
- Developing a "Hello World" with Jakarta Persistence API, Hibernate, and Spring Data
- Examining the configuration and integration options

In this chapter, we'll start with the Jakarta Persistence API (JPA), Hibernate, and Spring Data and work through a step-by-step example. We'll look at the persistence APIs and see the benefits of using either standardized JPA, native Hibernate, or Spring Data.

We'll begin with a tour through JPA, Hibernate, and Spring Data, looking at a straightforward "Hello World" application. JPA (Jakarta Persistence API, formerly Java Persistence API) is the specification defining an API that manages the persistence of objects and object/relational mappings—it specifies what must be done to persist objects. Hibernate, the most popular implementation of this specification, will make the persistence happen. Spring Data makes the implementation of the persistence layer even more efficient; it's an umbrella project that adheres to the Spring framework principles and offers an even simpler approach.

2.1 Introducing Hibernate

Object/relational mapping (ORM) is a programming technique for making the connection between the incompatible worlds of object-oriented systems and relational databases. Hibernate is an ambitious project that aims to provide a complete solution to the problem of managing persistent data in Java. Today, Hibernate is not only an ORM service but also a collection of data management tools extending well beyond ORM.

The Hibernate project suite includes the following:

- *Hibernate ORM*—Hibernate ORM consists of a core, a base service for persistence with SQL databases, and a native proprietary API. Hibernate ORM is the foundation for several of the other projects in the suite, and it's the oldest Hibernate project. You can use Hibernate ORM on its own, independent of any framework or any particular runtime environment with all JDKs. As long as a data source is accessible, you can configure it for Hibernate, and it works.

- *Hibernate EntityManager*—This is Hibernate's implementation of the standard Jakarta Persistence API. It's an optional module you can stack on top of Hibernate ORM. Hibernate's native features are a superset of the JPA persistence features in every respect.
- *Hibernate Validator*—Hibernate provides the reference implementation of the Bean Validation (JSR 303) specification. Independent of other Hibernate projects, it provides declarative validation for domain model (or any other) classes.
- *Hibernate Envers*—Envers is dedicated to audit logging and keeping multiple versions of data in the SQL database. This helps add data history and audit trails to the application, similar to any version control systems you might already be familiar with, such as Subversion or Git.
- *Hibernate Search*—Hibernate Search keeps an index of the domain model data up to date in an Apache Lucene database. It lets you query this database with a powerful and naturally integrated API. Many projects use Hibernate Search in addition to Hibernate ORM, adding full-text search capabilities. If you have a free text search form in your application's user interface, and you want happy users, work with Hibernate Search. Hibernate Search isn't covered in this book, but you can get a good start with *Hibernate Search in Action* by Emmanuel Bernard (Bernard, 2008).
- *Hibernate OGM*—This Hibernate project is an object/grid mapper. It provides JPA support for NoSQL solutions, reusing the Hibernate core engine but persisting mapped entities into key/value-, document-, or graph-oriented data stores.
- *Hibernate Reactive*—Hibernate Reactive is a reactive API for Hibernate ORM, interacting with a database in a non-blocking manner. It supports non-blocking database drivers. Hibernate Reactive isn't covered in this book.

The Hibernate source code is freely downloadable from **https://github.-com/hibernate**.

2.2 Introducing Spring Data

Spring Data is a family of projects belonging to the Spring framework whose purpose is to simplify access to both relational and NoSQL databases:

- *Spring Data Commons*—Spring Data Commons, part of the umbrella Spring Data project, provides a metadata model for persisting Java classes and technology-neutral repository interfaces.
- *Spring Data JPA*—Spring Data JPA deals with the implementation of JPA-based repositories. It provides improved support for JPA-based data access layers by reducing the boilerplate code and creating implementations for the repository interfaces.

- *Spring Data JDBC*—Spring Data JDBC deals with the implementation of JDBC-based repositories. It provides improved support for JDBC-based data access layers. It does not offer a series of JPA capabilities, such as caching or lazy loading, resulting in a simpler and limited ORM.
- *Spring Data REST*—Spring Data REST deals with exporting Spring Data repositories as RESTful resources.
- *Spring Data MongoDB*—Spring Data MongoDB deals with access to the MongoDB document database. It relies on the repository-style data access layer and the POJO programming model.
- *Spring Data Redis*—Spring Data Redis deals with access to the Redis key/value database. It relies on freeing the developer from managing the infrastructure and providing high- and low-level abstractions for access to the data store. Spring Data Redis isn't covered in this book.

The Spring Data source code (together with other Spring projects) is freely downloadable from **https://github.com/spring-projects**.

Let's get started with our first JPA, Hibernate, and Spring Data project.

2.3 "Hello World" with JPA

In this section we'll write our first JPA application, which will store a message in the database and then retrieve it. The machine we are running our code on has MySQL Release 8.0 installed. To install MySQL Release 8.0, follow the instructions in the official documentation: **https://dev.mysql.com/doc/refman/8.0/en/installing.html**.

In order to execute the examples in the source code, you'll need first to run the Ch02.sql script, as shown in figure 2.1. Open MySQL Workbench, go to File > Open SQL Script, and choose the SQL file and run it. The examples use a MySQL server with the default credentials: the username *root* and no password.

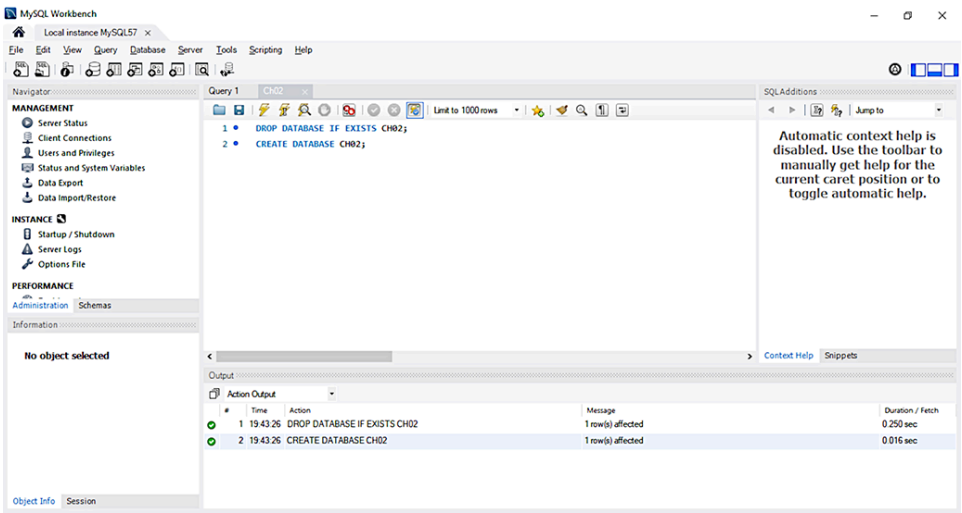

Figure 2.1 Creating the MySQL database by running the Ch02.sql script

In the “Hello World” application, we want to store messages in the database and load them from the database. Hibernate applications define persistent classes that are mapped to database tables. We define these classes based on our analysis of the business domain; hence, they’re a model of the domain. This example will consist of one class and its mapping. We’ll write the examples as executable tests, with assertions that verify the correct outcome of each operation. We’ve tested all the examples in this book, so we can be sure they work properly.

Let’s start by installing and configuring JPA, Hibernate, and the other needed dependencies. We’ll use Apache Maven as the project build tool for all the examples in this book. For basic Maven concepts and details on how to set up Maven, see appendix A.

We’ll declare the dependencies shown in the following listing.

Listing 2.1 The Maven dependencies on Hibernate, JUnit Jupiter, and MySQL

```
Path: Ch02/helloworld/pom.xml

<dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-entitymanager</artifactId>
    <version>5.6.9.Final</version>
</dependency>
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-engine</artifactId>
    <version>5.8.2</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.29</version>
</dependency>
```

The `hibernate-entitymanager` module includes transitive dependencies on other modules we’ll need, such as `hibernate-core` and the JPA interface stubs. We also need the `junit-jupiter-engine` dependency to run the tests with the help of JUnit 5, and the `mysql-connector-java` dependency, which is the official JDBC driver for MySQL.

Our starting point in JPA is the *persistence unit*. A persistence unit is a pairing of our domain model class mappings with a database connection, plus some other configuration settings. Every application has at least one persistence unit; some applications have several if they’re talking to sev-

eral (logical or physical) databases. Hence, our first step is setting up a persistence unit in our application's configuration.

2.3.1 Configuring a persistence unit

The standard configuration file for persistence units is located on the classpath in META-INF/persistence.xml. Create the following configuration file for the "Hello World" application.

Listing 2.2 The persistence.xml configuration file

```
Path: Ch02/helloworld/src/main/resources/META-INF/persistence.xml

<persistence xmlns="http://java.sun.com/xml/ns/persistence"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
             ➥ http://java.sun.com/xml/ns/persistence/persistence_2_0.x
             version="2.0">

    <persistence-unit name="ch02">
        <provider>org.hibernate.jpa.HibernatePersistenceProvider</provi
        <properties>
            <property name="javax.persistence.jdbc.driver"
                      value="com.mysql.cj.jdbc.Driver"/>
            <property name="javax.persistence.jdbc.url"
             value="jdbc:mysql://localhost:3306/CH02?serverTimezone=UTC
            <property name="javax.persistence.jdbc.user" value="root"/>
            <property name="javax.persistence.jdbc.password" value=""/>

            <property name="hibernate.dialect"
                   value="org.hibernate.dialect.MySQL8Dialect"/>

            <property name="hibernate.show_sql" value="true"/>
            <property name="hibernate.format_sql" value="true"/>

            <property name="hibernate.hbm2ddl.auto" value="create"/>
        </properties>
    </persistence-unit>

</persistence>
```

Ⓐ The persistence.xml file configures at least one persistence unit; each unit must have a unique name.

Ⓑ As JPA is only a specification, we need to indicate the vendor-specific PersistenceProvider implementation of the API. The persistence we define will be backed by a Hibernate provider.

Ⓒ Indicate the JDBC properties—the driver.

Ⓓ The URL of the database.

Ⓔ The username.

Ⓕ There is no password for access. The machine we are running the programs on has MySQL 8 installed, and the access credentials are the ones from persistence.xml. You should modify the credentials to correspond to the ones on your machine.

Ⓖ The Hibernate dialect is MySQL8, as the database to interact with is MySQL Release 8.0.

Ⓗ While executing, show the SQL code.

Ⓘ Hibernate will format the SQL nicely and generate comments in the SQL string so we know why Hibernate executed the SQL statement.

Ⓙ Every time the program is executed, the database will be created from scratch. This is ideal for automated testing, when we want to work with a clean database for every test run.

Let's see what a simple persistent class looks like, how the mapping is created, and some of the things we can do with instances of the persistent class in JPA.

2.3.2 Writing a persistent class

The objective of this example is to store messages in a database and retrieve them for display. The application has a simple persistent class, `Message`.

Listing 2.3 The `Message` class

```
Path: Ch02/helloworld/src/main/java/com/manning/javapersistence/ch02
➥ /Message.java

@Entity                                                              Ⓐ
public class Message {

    @Id                                                              Ⓑ
    @GeneratedValue(strategy = GenerationType.IDENTITY)              Ⓒ
    private Long id;

    private String text;                                             Ⓓ

    public String getText() {                                        Ⓓ
        return text;                                                 Ⓓ
    }                                                                Ⓓ

    public void setText(String text) {                               Ⓓ
```

```
        this.text = text;                                    Ⓓ
    }                                                        Ⓓ
}
```

Ⓐ Every persistent entity class must have at least the `@Entity` annotation. Hibernate maps this class to a table called `MESSAGE`.

Ⓑ Every persistent entity class must have an identifier attribute annotated with `@Id`. Hibernate maps this attribute to a column named `id`.

Ⓒ Someone must generate identifier values; this annotation enables automatic generation of ids.

Ⓓ We usually implement regular attributes of a persistent class with private fields and public getter/setter method pairs. Hibernate maps this attribute to a column called `text`.

The identifier attribute of a persistent class allows the application to access the database identity—the primary key value—of a persistent instance. If two instances of `Message` have the same identifier value, they represent the same row in the database. This example uses `Long` for the type of identifier attribute, but this isn't a requirement. Hibernate allows you to use virtually anything for the identifier type, as you'll see later in the book.

You may have noticed that the `text` attribute of the `Message` class has JavaBeans-style property accessor methods. The class also has a (default) constructor with no parameters. The persistent classes we'll show in the examples will usually look something like this. Note that we don't need to implement any particular interface or extend any special superclass.

Instances of the `Message` class can be managed (made persistent) by Hibernate, but they don't have to be. Because the `Message` object doesn't implement any persistence-specific classes or interfaces, we can use it just like any other Java class:

```
Message msg = new Message();
msg.setText("Hello!");
System.out.println(msg.getText());
```

It may look like we're trying to be cute here; in fact, we're demonstrating an important feature that distinguishes Hibernate from some other persistence solutions. We can use the persistent class in any execution context—*no special container is needed.*

We don't have to use annotations to map a persistent class. Later we'll show other mapping options, such as the JPA orm.xml mapping file and

the native hbm.xml mapping files, and we'll look at when they're a better solution than source annotations, which are the most frequently used approach nowadays.

The `Message` class is now ready. We can store instances in our database and write queries to load them again into application memory.

2.3.3 Storing and loading messages

What you really came here to see is JPA with Hibernate, so let's save a new `Message` to the database.

Listing 2.4 The `HelloWorldJPATest` class

```
Path: Ch02/helloworld/src/test/java/com/manning/javapersistence/ch02
➥ /HelloWorldJPATest.java

public class HelloWorldJPATest {

    @Test
    public void storeLoadMessage() {

        EntityManagerFactory emf =
                Persistence.createEntityManagerFactory("ch02");

        try {
            EntityManager em = emf.createEntityManager();
            em.getTransaction().begin();

            Message message = new Message();
            message.setText("Hello World!");

            em.persist(message);

            em.getTransaction().commit();
            //INSERT into MESSAGE (ID, TEXT) values (1, 'Hello World!')

            em.getTransaction().begin();


            List<Message> messages =
                em.createQuery("select m from Message m", Message.class
                  .getResultList();
            //SELECT * from MESSAGE

            messages.get(messages.size() - 1).
                     setText("Hello World from JPA!");

            em.getTransaction().commit();
            //UPDATE MESSAGE set TEXT = 'Hello World from JPA!'
            ➥ where ID = 1
```

```
            assertAll(
                    () -> assertEquals(1, messages.size()),
                    () -> assertEquals("Hello World from JPA!",
                                messages.get(0).getText())
            );

            em.close();

        } finally {
            emf.close();
        }
    }

}
```

Ⓐ First we need an `EntityManagerFactory` to talk to the database. This API represents the persistence unit, and most applications have one `EntityManager-Factory` for one configured persistence unit. Once it starts, the application should create the `EntityManagerFactory`; the factory is thread-safe, and all code in the application that accesses the database should share it.

Ⓑ Begin a new session with the database by creating an `EntityManager`. This is the context for all persistence operations.

Ⓒ Get access to the standard transaction API, and begin a transaction on this thread of execution.

Ⓓ Create a new instance of the mapped domain model class `Message`, and set its `text` property.

Ⓔ Enlist the transient instance with the persistence context; we make it persistent. Hibernate now knows that we wish to store that data, but it doesn't necessarily call the database immediately.

Ⓕ Commit the transaction. Hibernate automatically checks the persistence context and executes the necessary SQL `INSERT` statement. To help you understand how Hibernate works, we show the automatically generated and executed SQL statements in source code comments when they occur. Hibernate inserts a row in the `MESSAGE` table, with an automatically generated value for the `ID` primary key column, and the `TEXT` value.

Ⓖ Every interaction with the database should occur within transaction boundaries, even if we're only reading data, so we start a new transaction. Any potential failure appearing from now on will not affect the previously committed transaction.

Ⓗ Execute a query to retrieve all instances of `Message` from the database.

Ⓘ We can change the value of a property. Hibernate detects this automatically because the loaded `Message` is still attached to the persistence context it was loaded in.

Ⓙ On commit, Hibernate checks the persistence context for dirty state, and it executes the SQL `UPDATE` automatically to synchronize in-memory objects with the database state.

Ⓚ Check the size of the list of messages retrieved from the database.

Ⓛ Check that the message we persisted is in the database. We use the JUnit 5 `assertAll` method, which always checks all the assertions that are passed to it, even if some of them fail. The two assertions that we verify are conceptually related.

Ⓜ We created an `EntityManager`, so we must close it.

Ⓝ We created an `EntityManagerFactory`, so we must close it.

The query language you've seen in this example isn't SQL, it's the Jakarta Persistence Query Language (JPQL). Although there is syntactically no difference in this trivial example, the `Message` in the query string doesn't refer to the database table name but to the persistent class name. For this reason, the `Message` class name in the query is case-sensitive. If we map the class to a different table, the query will still work.

Also, notice how Hibernate detects the modification to the text property of the message and automatically updates the database. This is the automatic dirty-checking feature of JPA in action. It saves us the effort of explicitly asking the persistence manager to update the database when we modify the state of an instance inside a transaction.

Figure 2.2 shows the result of checking for the existence of the record we inserted and updated on the database side. As you'll recall, we created a database named CH02 by running the Ch02.sql script from the chapter's source code.

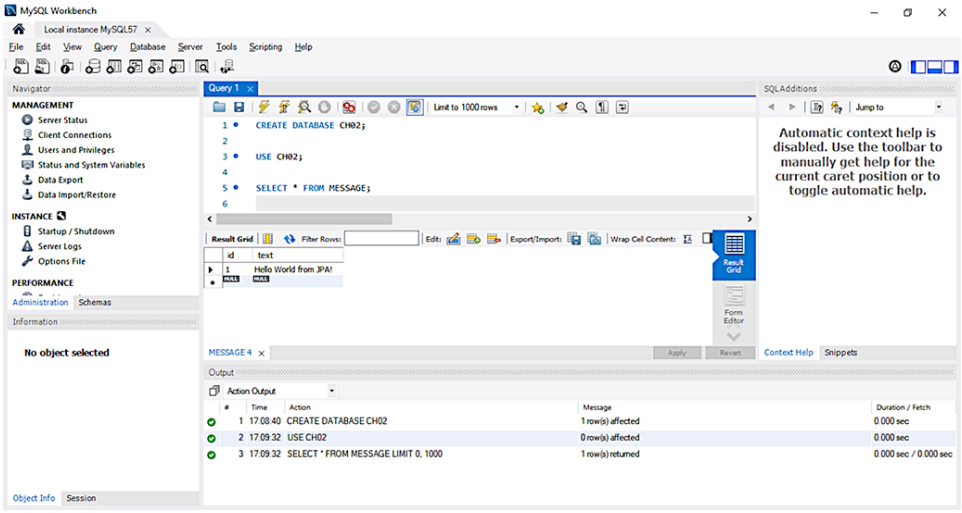

Figure 2.2 The result of checking for the existence of the inserted and updated record on the database side

You've just completed your first JPA and Hibernate application. Let's now take a quick look at the native Hibernate bootstrap and configuration API.

2.4 Native Hibernate configuration

Although basic (and extensive) configuration is standardized in JPA, we can't access all the configuration features of Hibernate with properties in persistence.xml. Note that most applications, even quite sophisticated ones, don't need such special configuration options and hence don't have to access the bootstrap API we'll show in this section. If you aren't sure, you can skip this section and come back to it later when you need to extend Hibernate type adapters, add custom SQL functions, and so on.

When using native Hibernate we'll use the Hibernate dependencies and API directly, rather than the JPA dependencies and classes. JPA is a specification, and it can use different implementations (Hibernate is one example, but EclipseLink is another alternative) through the same API. Hibernate, as an implementation, provides its own dependencies and classes. While using JPA provides more flexibility, you'll see throughout the book that accessing the Hibernate implementation directly allows you to use features that are not covered by the JPA standard (we'll point this out where it's relevant).

The native equivalent of the standard JPA `EntityManagerFactory` is the `org.hibernate.SessionFactory`. We have usually one per application, and it involves the same pairing of class mappings with database connection configuration.

To configure the native Hibernate, we can use a hibernate.properties Java properties file or a hibernate.cfg.xml XML file. We'll choose the second option, and the configuration will contain database and session-related options. This XML file is generally placed in the src/main/resource or

src/test/resource folder. As we need the information for the Hibernate configuration in our tests, we'll choose the second location.

Listing 2.5 The hibernate.cfg.xml configuration file

```
Path: Ch02/helloworld/src/test/resources/hibernate.cfg.xml

<?xml version='1.0' encoding='utf-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD//EN"
➥ "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property name="hibernate.connection.driver_class">
            com.mysql.cj.jdbc.Driver
        </property>
        <property name="hibernate.connection.url">
            jdbc:mysql://localhost:3306/CH02?serverTimezone=UTC
        </property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password"></property>
        <property name="hibernate.connection.pool_size">50</property>
        <property name="show_sql">true</property>
        <property name="hibernate.hbm2ddl.auto">create</property>
    </session-factory>
</hibernate-configuration>
```

Ⓐ We use the tags to indicate that we are configuring Hibernate.

Ⓑ More exactly, we are configuring the `SessionFactory` object. `SessionFactory` is an interface, and we need one `SessionFactory` to interact with one database.

Ⓒ Indicate the JDBC properties—the driver.

Ⓓ The URL of the database.

Ⓔ The username.

Ⓕ No password is required to access it. The machine we are running the programs on has MySQL 8 installed and the access credentials are the ones from hibernate .cfg.xml. You should modify the credentials to correspond to the ones on your machine.

Ⓖ Limit the number of connections waiting in the Hibernate database connection pool to 50.

Ⓗ While executing, the SQL code is shown.

Ⓘ Every time the program is executed, the database will be created from scratch. This is ideal for automated testing, when we want to work with a clean database for every test run.

Let's save a `Message` to the database using native Hibernate.

Listing 2.6 The `HelloWorldHibernateTest` class

```
Path: Ch02/helloworld/src/test/java/com/manning/javapersistence/ch02
➥ /HelloWorldHibernateTest.java

public class HelloWorldHibernateTest {

    private static SessionFactory createSessionFactory() {
        Configuration configuration = new Configuration();
        configuration.configure().addAnnotatedClass(Message.class);
        ServiceRegistry serviceRegistry = new
                StandardServiceRegistryBuilder().
                applySettings(configuration.getProperties()).build();
        return configuration.buildSessionFactory(serviceRegistry);
    }

    @Test
    public void storeLoadMessage() {

        try (SessionFactory sessionFactory = createSessionFactory();
             Session session = sessionFactory.openSession()) {

            session.beginTransaction();

            Message message = new Message();
            message.setText("Hello World from Hibernate!");

            session.persist(message);

            session.getTransaction().commit();
            // INSERT into MESSAGE (ID, TEXT)
            // values (1, 'Hello World from Hibernate!')
            session.beginTransaction();

            CriteriaQuery<Message> criteriaQuery =
                 session.getCriteriaBuilder().createQuery(Message.class
            criteriaQuery.from(Message.class);

            List<Message> messages =
                 session.createQuery(criteriaQuery).getResultList();
            // SELECT * from MESSAGE

            session.getTransaction().commit();

            assertAll(
                    () -> assertEquals(1, messages.size()),
                    () -> assertEquals("Hello World from Hibernate!",
```

```
                                        messages.get(0).getText())
                );

            }
        }
    }
```

Ⓐ To create a `SessionFactory`, we first need to create a configuration.

Ⓑ We need to call the `configure` method on it and to add `Message` to it as an annotated class. The execution of the `configure` method will load the content of the default hibernate.cfg.xml file.

Ⓒ The builder pattern helps us create the immutable service registry and configure it by applying settings with chained method calls. A `ServiceRegistry` hosts and manages services that need access to the `SessionFactory`. Services are classes that provide pluggable implementations of different types of functionality to Hibernate.

Ⓓ Build a `SessionFactory` using the configuration and the service registry we have previously created.

Ⓔ The `SessionFactory` created with the `createSessionFactory` method we previously defined is passed as an argument to a `try` with resources, as `SessionFactory` implements the `AutoCloseable` interface.

Ⓕ Similarly, we begin a new session with the database by creating a `Session`, which also implements the `AutoCloseable` interface. This is our context for all persistence operations.

Ⓕ Get access to the standard transaction API and begin a transaction on this thread of execution.

Ⓗ Create a new instance of the mapped domain model class `Message`, and set its `text` property.

Ⓘ Enlist the transient instance with the persistence context; we make it persistent. Hibernate now knows that we wish to store that data, but it doesn't necessarily call the database immediately. The native Hibernate API is pretty similar to the standard JPA, and most methods have the same name.

Ⓙ Synchronize the session with the database, and close the current session on commit of the transaction automatically.

Ⓚ Begin another transaction. Every interaction with the database should occur within transaction boundaries, even if we're only reading data.

Ⓛ Create an instance of `CriteriaQuery` by calling the `CriteriaBuilder createQuery()` method. A `CriteriaBuilder` is used to construct criteria queries, compound selections, expressions, predicates, and orderings. A `CriteriaQuery` defines functionality that is specific to top-level queries. `CriteriaBuilder` and `CriteriaQuery` belong to the Criteria API, which allows us to build a query programmatically.

Ⓜ Create and add a query root corresponding to the given `Message` entity.

Ⓝ Call the `getResultList()` method of the query object to get the results. The query that is created and executed will be `SELECT * FROM MESSAGE`.

Ⓞ Commit the transaction.

Ⓟ Check the size of the list of messages retrieved from the database.

Ⓠ Check that the message we persisted is in the database. We use the JUnit 5 `assertAll` method, which always checks all the assertions that are passed to it, even if some of them fail. The two assertions that we verify are conceptually related.

Figure 2.3 shows the result of checking for the existence of the record we inserted on the database side by using native Hibernate.

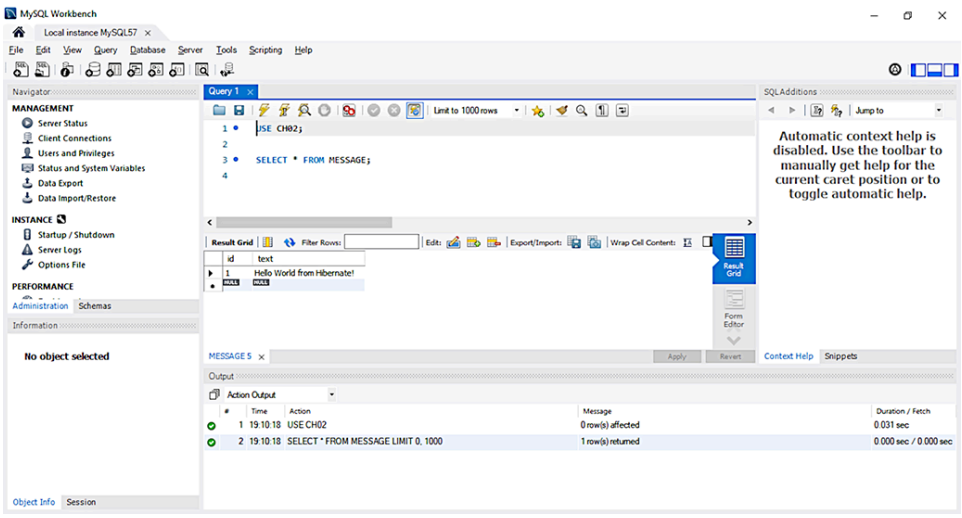

Figure 2.3 The result of checking for the existence of the inserted record on the database side

Most of the examples in this book won't use the `SessionFactory` or `Session` API. From time to time, when a particular feature is only available in Hibernate, we'll show you how to `unwrap()` the native interface.

2.5 Switching between JPA and Hibernate

Suppose you're working with JPA and need to access the Hibernate API. Or, vice versa, you're working with native Hibernate and you need to create an `EntityManagerFactory` from the Hibernate configuration. To obtain a `SessionFactory` from an `EntityManagerFactory`, you'll have to unwrap the first one from the second one.

Listing 2.7 Obtaining a `SessionFactory` from an `EntityManagerFactory`

```
Path: Ch02/helloworld/src/test/java/com/manning/javapersistence/ch02
➥ /HelloWorldJPAToHibernateTest.java

private static SessionFactory getSessionFactory
                (EntityManagerFactory entityManagerFactory) {
    return entityManagerFactory.unwrap(SessionFactory.class);
}
```

Starting with JPA version 2.0, you can get access to the APIs of the underlying implementations. The `EntityManagerFactory` (and also the `EntityManager`) declares an `unwrap` method that will return objects belonging to the classes of the JPA implementation. When using the Hibernate implementation, you can get the corresponding `SessionFactory` or `Session` objects and start using them as demonstrated in listing 2.6. When a particular feature is only available in Hibernate, you can switch to it using the `unwrap` method.

You may be interested in the reverse operation: creating an `EntityManagerFactory` from an initial Hibernate configuration.

Listing 2.8 Obtaining an `EntityManagerFactory` from a Hibernate configuration

```
Path: Ch02/helloworld/src/test/java/com/manning/javapersistence/ch02
➥ /HelloWorldHibernateToJPATest.java

private static EntityManagerFactory createEntityManagerFactory() {
    Configuration configuration = new Configuration();
    configuration.configure().addAnnotatedClass(Message.class);

    Map<String, String> properties = new HashMap<>();
    Enumeration<?> propertyNames =
              configuration.getProperties().propertyNames();
    while (propertyNames.hasMoreElements()) {
        String element = (String) propertyNames.nextElement();
        properties.put(element,
            configuration.getProperties().getProperty(element));
    }
```

```
        return Persistence.createEntityManagerFactory("ch02", properties);
    }
```

Ⓐ Create a new Hibernate configuration.

Ⓑ Call the `configure` method, which adds the content of the default hibernate .cfg.xml file to the configuration, and then explicitly add `Message` as an annotated class.

Ⓒ Create a new hash map to be filled in with the existing properties.

Ⓓ Get all the property names from the Hibernate configuration.

Ⓔ Add the property names one by one to the previously created map.

Ⓕ Return a new `EntityManagerFactory` , providing to it the ch02.ex01 persistence unit name and the previously created map of properties.

2.6 “Hello World” with Spring Data JPA

Let’s now write our first Spring Data JPA application, which will store a message in the database and then retrieve it.

We’ll first add the Spring dependencies to the Apache Maven configuration.

Listing 2.9 The Maven dependencies on Spring

```
Path: Ch02/helloworld/pom.xml

<dependency>                                                    Ⓐ
    <groupId>org.springframework.data</groupId>                 Ⓐ
    <artifactId>spring-data-jpa</artifactId>                    Ⓐ
    <version>2.7.0</version>                                    Ⓐ
</dependency>                                                   Ⓐ
<dependency>                                                    Ⓑ
    <groupId>org.springframework</groupId>                      Ⓑ
    <artifactId>spring-test</artifactId>                        Ⓑ
    <version>5.3.20</version>                                   Ⓑ
</dependency>                                                   Ⓑ
```

Ⓐ The `spring-data-jpa` module provides repository support for JPA and includes transitive dependencies on other modules we’ll need, such as `spring-core` and `spring-context` .

Ⓑ We also need the `spring-test` dependency to run the tests.

The standard configuration file for Spring Data JPA is a Java class that creates and sets up the beans needed by Spring Data. The configuration can be done using either an XML file or Java code, and we've chosen the second alternative. Create the following configuration file for the "Hello World" application.

Listing 2.10 The `SpringDataConfiguration` class

```
Path: Ch02/helloworld/src/test/java/com/manning/javapersistence/ch02
➥ /configuration/SpringDataConfiguration.java

@EnableJpaRepositories("com.manning.javapersistence.ch02.repositories")
public class SpringDataConfiguration {
    @Bean
    public DataSource dataSource() {
        DriverManagerDataSource dataSource = new DriverManagerDataSourc
        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
        dataSource.setUrl(
           "jdbc:mysql://localhost:3306/CH02?serverTimezone=UTC");
        dataSource.setUsername("root");
        dataSource.setPassword("");
        return dataSource;
    }

    @Bean
    public JpaTransactionManager
           transactionManager(EntityManagerFactory emf) {
        return new JpaTransactionManager(emf);
    }

    @Bean
    public JpaVendorAdapter jpaVendorAdapter() {
        HibernateJpaVendorAdapter jpaVendorAdapter = new
                 HibernateJpaVendorAdapter();
        jpaVendorAdapter.setDatabase(Database.MYSQL);
        jpaVendorAdapter.setShowSql(true);
        return jpaVendorAdapter;
    }

    @Bean
    public LocalContainerEntityManagerFactoryBean entityManagerFactory(
        LocalContainerEntityManagerFactoryBean
          localContainerEntityManagerFactoryBean =
                new LocalContainerEntityManagerFactoryBean();
        localContainerEntityManagerFactoryBean.setDataSource(dataSource
        Properties properties = new Properties();
        properties.put("hibernate.hbm2ddl.auto", "create");
        localContainerEntityManagerFactoryBean.
                setJpaProperties(properties);
        localContainerEntityManagerFactoryBean.
                setJpaVendorAdapter(jpaVendorAdapter());
        localContainerEntityManagerFactoryBean.
                setPackagesToScan("com.manning.javapersistence.ch02");
```

```
            return localContainerEntityManagerFactoryBean;
        }
    }
```

Ⓐ The `@EnableJpaRepositories` annotation enables scanning of the package received as an argument for Spring Data repositories.

Ⓑ Create a data source bean.

Ⓒ Specify the JDBC properties—the driver.

Ⓓ The URL of the database.

Ⓔ The username.

Ⓕ No password is required for access. The machine we are running the programs on has MySQL 8 installed, and the access credentials are the ones from this configuration. You should modify the credentials to correspond to the ones on your machine.

Ⓖ Create a transaction manager bean based on an entity manager factory. Every interaction with the database should occur within transaction boundaries, and Spring Data needs a transaction manager bean.

Ⓗ Create the JPA vendor adapter bean needed by JPA to interact with Hibernate.

Ⓘ Configure this vendor adapter to access a MySQL database.

Ⓙ Show the SQL code while it is executed.

Ⓚ Create a `LocalContainerEntityManagerFactoryBean`. This is a factory bean that produces an `EntityManagerFactory` following the JPA standard container bootstrap contract.

Ⓛ Set the data source.

Ⓜ Set the database to be created from scratch every time the program is executed.

Ⓝ Set the vendor adapter.

Ⓞ Set the packages to scan for entity classes. As the `Message` entity is located in `com.manning.javapersistence.ch02`, we set this package to be scanned.

Spring Data JPA provides support for JPA-based data access layers by reducing the boilerplate code and creating implementations for the reposi-

tory interfaces. We only need to define our own repository interface to extend one of the Spring Data interfaces.

Listing 2.11 The `MessageRepository` interface

```
Path: Ch02/helloworld/src/main/java/com/manning/javapersistence/ch02
➥ /repositories/MessageRepository.java

public interface MessageRepository extends CrudRepository<Message, Long

}
```

The `MessageRepository` interface extends `CrudRepository <Message, Long>`. This means that it is a repository of `Message` entities with a `Long` identifier. Remember, the `Message` class has an `id` field annotated as `@Id` of type `Long`. We can directly call methods such as `save`, `findAll`, or `findById`, which are inherited from `CrudRepository`, and we can use them without any other additional information to execute the usual operations against a database. Spring Data JPA will create a proxy class implementing the `MessageRepository` interface and implement its methods (figure 2.4).

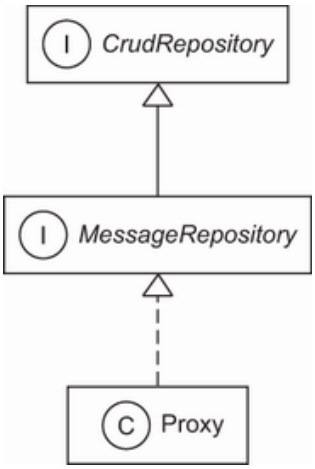

Figure 2.4 The Spring Data JPA `Proxy` class implements the `MessageRepository` interface.

Let's save a `Message` to the database using Spring Data JPA.

Listing 2.12 The `HelloWorldSpringDataJPATest` class

```
Path: Ch02/helloworld/src/test/java/com/manning/javapersistence/ch02
➥ /HelloWorldSpringDataJPATest.java

@ExtendWith(SpringExtension.class)
@ContextConfiguration(classes = {SpringDataConfiguration.class})
public class HelloWorldSpringDataJPATest {

    @Autowired
    private MessageRepository messageRepository;
```

```
    @Test
    public void storeLoadMessage() {
        Message message = new Message();
        message.setText("Hello World from Spring Data JPA!");

        messageRepository.save(message);

        List<Message> messages =
        ➥ (List<Message>)messageRepository.findAll();

        assertAll(
                () -> assertEquals(1, messages.size()),
                () -> assertEquals("Hello World from Spring Data JPA!",
                                   messages.get(0).getText())
        );

    }
}
```

Ⓐ Extend the test using `SpringExtension`. This extension is used to integrate the Spring test context with the JUnit 5 Jupiter test.

Ⓑ The Spring test context is configured using the beans defined in the previously presented `SpringDataConfiguration` class.

Ⓒ A `MessageRepository` bean is injected by Spring through autowiring. This is possible as the `com.manning.javapersistence.ch02.repositories` package where `MessageRepository` is located was used as the argument of the `@EnableJpaRepositories` annotation in listing 2.8. If we call `messageRepository.getClass()`, we'll see that it returns something like `com.sun.proxy.$Proxy41` —a proxy generated by Spring Data, as explained in figure 2.4.

Ⓓ Create a new instance of the mapped domain model class `Message`, and set its `text` property.

Ⓔ Persist the `message` object. The `save` method is inherited from the `CrudRepository` interface, and its body will be generated by Spring Data JPA when the proxy class is created. It will simply save a `Message` entity to the database.

Ⓕ Retrieve the messages from the repository. The `findAll` method is inherited from the `CrudRepository` interface, and its body will be generated by Spring Data JPA when the proxy class is created. It will simply return all entities belonging to the `Message` class.

Ⓖ Check the size of the list of messages retrieved from the database and that the message we persisted is in the database.

Ⓗ Use the JUnit 5 `assertAll` method, which checks all the assertions that are passed to it, even if some of them fail. The two assertions that we verify are conceptually related.

You'll notice that the Spring Data JPA test is considerably shorter than the ones using JPA or native Hibernate. This is because the boilerplate code has been removed—there's no more explicit object creation or explicit control of the transactions. The repository object is injected, and it provides the generated methods of the proxy class. The burden is now heavier on the configuration side, but this should be done only once per application.

Figure 2.5 shows the result of checking that the record we inserted with Spring Data JPA exists in the database.

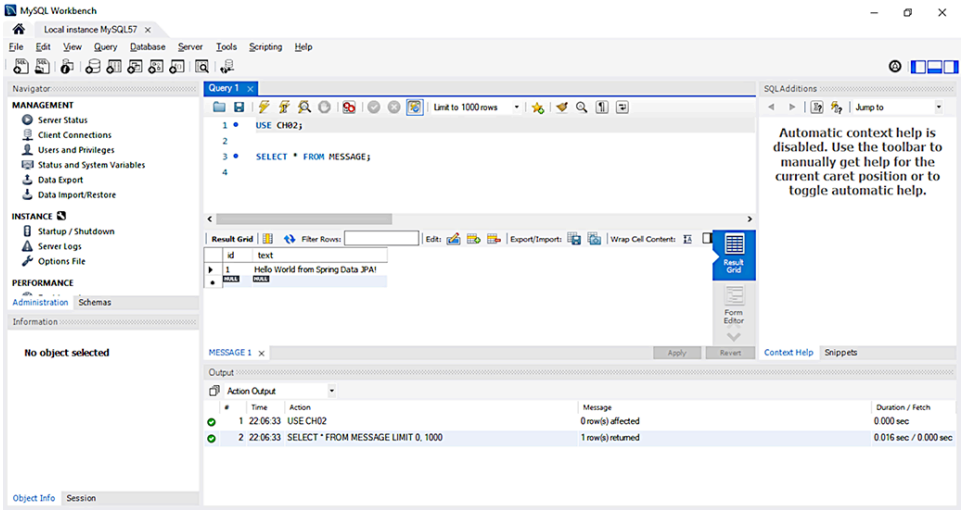

Figure 2.5 The result of checking that the inserted record exists in the database

2.7 Comparing the approaches of persisting entities

We have implemented a simple application that interacts with a database and uses, alternatively, JPA, native Hibernate, and Spring Data JPA. Our purpose was to analyze each approach and see how the configuration and code varies. Table 2.1 summarizes the characteristics of these approaches.

Table 2.1 Comparison of working with JPA, native Hibernate, and Spring

Data JPA

Framework	Characteristics
JPA	• Uses the general JPA API and requires a persistence provider. • We can switch between persistence providers from the configuration. • Requires explicit management of the `EntityManagerFactory`, `EntityManager`, and transactions. • The configuration and the amount of code to be written is similar to the native Hibernate native approach. • We can switch to the JPA approach by constructing an `EntityManagerFactory` from a native Hibernate configuration.
Native Hibernate	• Uses the native Hibernate API. You are locked into using this chosen framework. • Builds its configuration starting with the default Hibernate configuration files (hibernate.cfg.xml or hibernate.properties). • Requires explicit management of the `SessionFactory`, `Session`, and transactions. • The configuration and the amount of code to be written are similar to the JPA approach. • We can switch to the native Hibernate native approach by unwrapping a `Session-Factory` from an `EntityManagerFactory` or a `Session` from an `EntityManager`.
Spring Data JPA	• Needs additional Spring Data dependencies in the project. • The configuration will also take care of the creation of beans needed for the project, including the transaction manager. • The repository interface only needs to be declared, and Spring Data will create an implementation for it as a proxy class with generated methods that interact with the database.

- The necessary repository is injected and not explicitly created by the programmer.
- This approach requires the least amount of code to be written, as the configuration takes care of most of the burden.

For more information on the performance of each approach, see the article “Object-Relational Mapping Using JPA, Hibernate and Spring Data JPA,” by Cătălin Tudose and Carmen Odubășteanu (Tudose, 2021).

To analyze the running times, we executed a batch of insert, update, select, and delete operations using the three approaches, progressively increasing the number of records from 1,000 to 50,000. Tests were made on Windows 10 Enterprise, running on a four-core Intel i7-5500U processor at 2.40 GHz with 8 GB of RAM.

The insert execution times of Hibernate and JPA are very close (see table 2.2 and figure 2.6). The execution time for Spring Data JPA increases much faster with the increase of the number of records.

Table 2.2 Insert execution times by framework (times in ms)

Number of records	Hibernate	JPA	Spring Data JPA
1,000	1,138	1,127	2,288
5,000	3,187	3,307	8,410
10,000	5,145	5,341	14,565
20,000	8,591	8,488	26,313
30,000	11,146	11,859	37,579
40,000	13,011	13,300	48,913
50,000	16,512	16,463	59,629

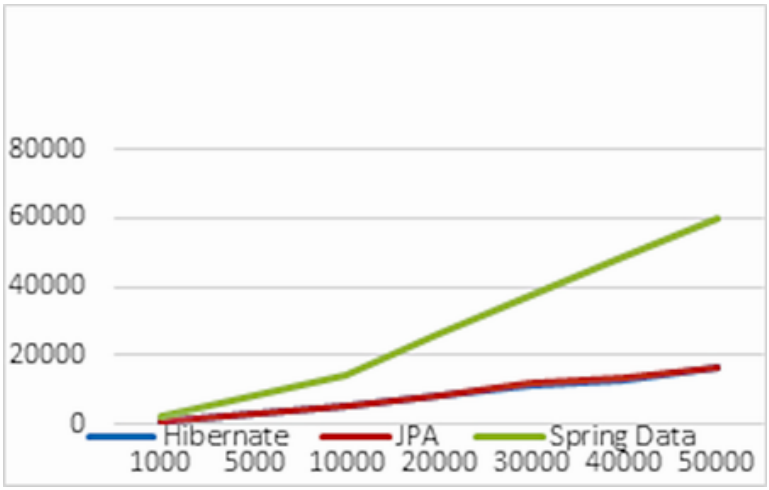

Figure 2.6 Insert execution times by framework (times in ms)

The update execution times of Hibernate and JPA are also very close (see table 2.3 and figure 2.7). Again, the execution time for Spring Data JPA increases much faster with the increase of the number of records.

Table 2.3 Update execution times by framework (times in ms)

Number of records	Hibernate	JPA	Spring Data JPA
1,000	706	759	2,683
5,000	2,081	2,256	10,211
10,000	3,596	3,958	17,594
20,000	6,669	6,776	33,090
30,000	9,352	9,696	46,341
40,000	12,720	13,614	61,599
50,000	16,276	16,355	75,071

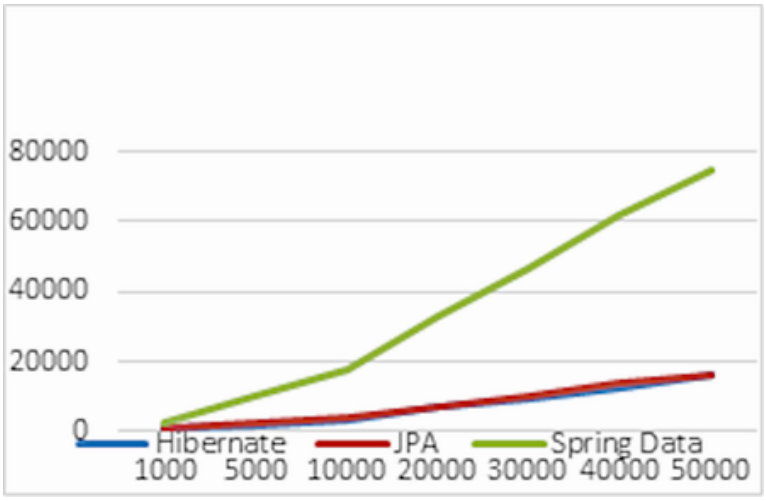

Figure 2.7 Update execution times by framework (times in ms)

The situation is similar for the select operations, with practically no difference between Hibernate and JPA, and a steep curve for Spring Data as the number of records increases (see table 2.4 and figure 2.8).

Table 2.4 Select execution times by framework (times in ms)

Number of records	Hibernate	JPA	Spring Data JPA
1,000	1,138	1,127	2,288
5,000	3,187	3,307	8,410
10,000	5,145	5,341	14,565
20,000	8,591	8,488	26,313
30,000	11,146	11,859	37,579
40,000	13,011	13,300	48,913
50,000	16,512	16,463	59,629

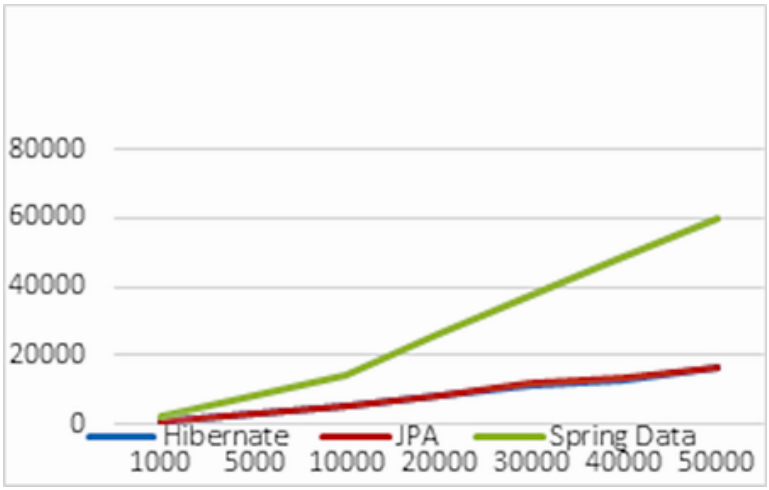

Figure 2.8 Select execution times by framework (times in ms)

It's no surprise that the delete operation behaves similarly, with Hibernate and JPA being close to each other, and Spring Data's execution time increasing faster as the number of records increases (see table 2.5 and figure 2.9).

Table 2.5 Delete execution times by framework (times in ms)

Number of records	Hibernate	JPA	Spring Data JPA
1,000	584	551	2,430
5,000	1,537	1,628	9,685
10,000	2,992	2,763	17,930
20,000	5,344	5,129	32,906
30,000	7,478	7,852	47,400
40,000	10,061	10,493	62,422
50,000	12,857	12,768	79,799

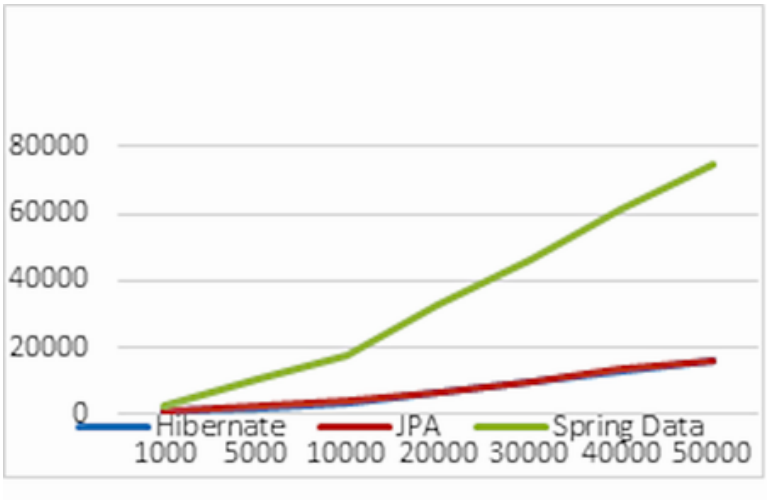

Figure 2.9 Delete execution times by framework (times in ms)

The three approaches provide different performances. Hibernate and JPA go head to head—the graphics of their times almost overlap for all four operations (insert, update, select, and delete). Even though JPA comes with its own API on top of Hibernate, this additional layer introduces no overhead.

The execution times of Spring Data JPA insertions start at about 2 times those of Hibernate and JPA for 1,000 records and go to about 3.5 times more for 50,000 records. The overhead of the Spring Data JPA framework is considerable.

For Hibernate and JPA, the update and delete execution times are lower than the insert execution times. In contrast, the Spring Data JPA update and delete execution times are longer than the insert execution times.

For Hibernate and JPA, the select times grow very slowly with the number of rows. The Spring Data JPA select execution times strongly grow with the number of rows.

Using Spring Data JPA is mainly justified in particular situations: if the project already uses the Spring framework and needs to rely on its existing paradigm (such as inversion of control or automatically managed transactions), or if there is a strong need to decrease the amount of code and thus shorten the development time (nowadays it is cheaper to acquire more computing power than to acquire more developers).

This chapter has focused on alternatives for working with databases from a Java application—JPA, native Hibernate, and Spring Data JPA—and we looked at introductory examples for each of them. Chapter 3 will introduce a more complex example and will deal in more depth with domain models and metadata.

Summary

- A Java persistence project can be implemented using three alternatives: JPA, native Hibernate, and Spring Data JPA.
- You can create, map, and annotate a persistent class.
- Using JPA, you can implement the configuration and bootstrapping of a persistence unit, and you can create the `EntityManagerFactory` entry point.
- You can call the `EntityManager` to interact with the database, storing and loading an instance of the persistent domain model class.
- Native Hibernate provides bootstrapping and configuration options, as well as the equivalent basic Hibernate APIs: `SessionFactory` and `Session`.
- You can switch between the JPA approach and Hibernate approach by unwrapping a `SessionFactory` from an `EntityManagerFactory` or obtaining an `EntityManagerFactory` from a Hibernate configuration.
- You can implement the configuration of a Spring Data JPA application by creating the repository interface, and then use it to store and load an instance of the persistent domain model class.
- Comparing and contrasting these three approaches (JPA, native Hibernate, and Spring Data JPA) demonstrates the advantages and shortcomings of each of them, in terms of portability, needed dependencies, amount of code, and execution speed.